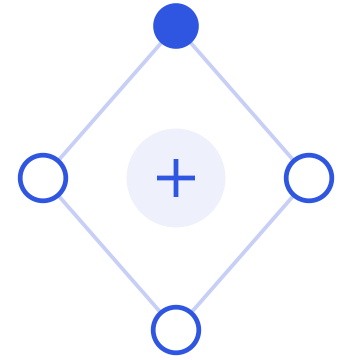


GUÍA PRÁCTICA · NEIRO AGENCY

Codex Plugin: metele a Codex a revisar tu código dentro de Claude Code



OpenAI sacó un plugin oficial que mete a Codex directo dentro de Claude Code, para que un segundo modelo (con otro entrenamiento y otros sesgos) revise el trabajo del primero sin salir de la terminal. Esta guía explica qué es, cómo instalarlo, y cómo usarlo sin que se coma tu cuota de uso.

Marcos Vargas · IA · Sistemas · Automatización

LO QUE APRENDERÁS

- 01 Qué es el Codex Plugin y por qué lo sacó OpenAI
- 02 Requisitos y cómo instalarlo, paso a paso
- 03 Los tres modos: review, adversarial-review y rescue
- 04 Los comandos completos, para manejar el trabajo en segundo plano
- 05 Cómo configurar el modelo (con la línea más nueva de OpenAI)
- 06 El "gate" automático: la función más peligrosa si no la cuidás
- 07 Cómo lo uso yo en el día a día, con mi propia experiencia

SECCIÓN 01
EMPECEMOS POR AQUÍ

Un segundo modelo revisando al primero, sin salir de la terminal.

El Codex Plugin para Claude Code es un plugin oficial de OpenAI que conecta Codex (el agente de código de ChatGPT) directo dentro de Claude Code. No levanta un motor aparte: usa el CLI de Codex y su app server, ya instalados en tu máquina, con la misma autenticación, configuración y servidores MCP que tengas configurados para Codex. En la práctica, agrega un puñado de comandos `/codex:` a Claude Code que te dejan pedirle a Codex que revise lo que hiciste, o delegarle una tarea completa, sin cambiar de programa.

La idea de fondo, y por qué vale la pena: el auto-repaso agarra errores de sintaxis. Casi nunca agarra problemas de diseño. Un modelo entrenado distinto, con sesgos distintos, revisando el mismo código, encuentra puntos ciegos que el modelo que lo escribió no ve, porque no está buscando sus propios errores de enfoque.

SECCIÓN 02
POR QUÉ EXISTE

OpenAI lo mete adentro de la herramienta que ya usás, no al revés.

Claude Code domina hoy el mercado de agentes de código. En vez de pedirle a los desarrolladores que se cambien de herramienta, OpenAI optó por meter a Codex directo en el flujo que la gente ya tiene armado: instalás un plugin, y de repente tenés acceso a Codex sin abrir una segunda terminal ni memorizar un segundo set de comandos. Algunos medios lo leyeron como una jugada de "caballo de Troya": la apuesta es que usar Codex todos los días, aunque sea de este lado, termina acostumbrando al desarrollador al ecosistema de OpenAI.

Para quien lo usa, la lectura estratégica importa menos que el resultado práctico: tenés acceso a una segunda opinión con un modelo distinto, gratis en términos de fricción, usando la cuota que ya pagás en tu plan de ChatGPT o tu API key de OpenAI.

SECCIÓN 03 REQUISITOS

Necesitás cuenta de OpenAI y Node. Nada más raro que eso.

- **Suscripción de ChatGPT** (el plan gratis también sirve) **o una API key de OpenAI**. El uso del plugin consume tu cuota de Codex, la misma que usarías desde la app o el CLI directo.
- **Node.js 18.18 o superior** instalado en tu máquina.
- Si ya usás Codex y estás logueado en esta compu, esa misma sesión sirve para el plugin: no hace falta una cuenta aparte.

SECCIÓN 04 CÓMO INSTALARLO

Cuatro comandos adentro de Claude Code.

Todo se instala desde dentro de una sesión de Claude Code:

INSTALACIÓN DEL PLUGIN

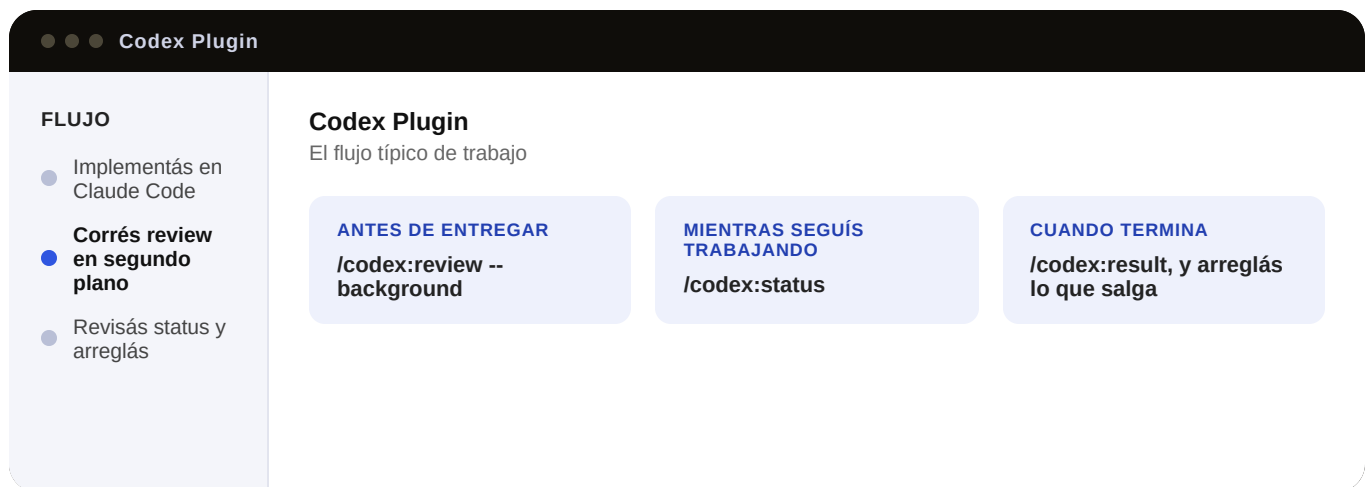
```
/plugin marketplace add openai/codex-plugin-cc  
/plugin install codex@openai-codex  
/reload-plugins  
/codex:setup
```

`/codex:setup` revisa si Codex está listo, y si falta y tenés npm, se ofrece a instalarlo por vos. Si preferís instalarlo a mano: `npm install -g @openai/codex`, y si ya está instalado pero no logueado: `!codex login` (acepta tanto cuenta de ChatGPT como API key). Un primer uso simple para probar que quedó bien: `/codex:review --background`, después `/codex:status` y `/codex:result`.

SECCIÓN 05
LOS COMANDOS

Siete comandos, dos que revisan y el resto que delegan y administran.

- **/codex:review:** revisión de solo lectura sobre tus cambios sin committear, o contra una rama base con `--base <ref>`. No es dirigible ni acepta texto de enfoque.
- **/codex:adversarial-review:** revisión dirigible que cuestiona la decisión de diseño, no solo el código: le podés indicar en qué enfocarse (conurrencia, rollback, seguridad).
- **/codex:rescue:** le delega una tarea completa a Codex (investigar un bug, probar un fix, retomar un intento anterior), con `--model` y `--effort` para elegir modelo y nivel de razonamiento.
- **/codex:transfer:** convierte la sesión actual de Claude Code en un hilo persistente de Codex, para seguirla desde ahí.
- **/codex:status** y **/codex:result:** revisan el avance de un trabajo en segundo plano y muestran el resultado final.
- **/codex:cancel:** corta un trabajo en segundo plano que ya no necesitás.



SECCIÓN 06 CONFIGURACIÓN

El modelo y el esfuerzo se ajustan en un archivo, no en cada comando.

El plugin respeta la configuración normal de Codex. Se sobrescribe con un `config.toml`, a nivel de usuario (`~/.codex/config.toml`) o de proyecto (`.codex/config.toml`), y solo se aplica si el proyecto está marcado como de confianza):

EN CONFIG.TOML

```
model = "gpt-6-sol"
model_reasoning_effort = "high"
```

OpenAI maneja su línea de modelos en tres niveles (Astra, Sol y Luna), y los nombres disponibles cambian seguido: confirmá cuáles tenés con `/model` dentro de Codex. Lo mismo aplica a los planes y la cuota de ChatGPT: varían según la región y cambian muy rápido, así que verificalos en la web oficial de OpenAI. En general, **Luna** es la opción rápida y barata, **Sol** el punto medio para revisión de código del día a día, y **Astra** la más capaz y más cara. Para `/codex:rescue` también existe la opción `--model spark`, que manda la tarea al modelo más instantáneo de la línea Codex, pensado para arreglos chicos y rápidos.

SECCIÓN 07 EL GATE AUTOMÁTICO

Existe, pero no lo actives sin estar viendo la sesión.

Con `/codex:setup --enable-review-gate` se activa un modo donde, cada vez que Claude termina una respuesta, un hook corre una revisión de Codex automática, y si encuentra problemas bloquea que la sesión pare hasta que se corrijan. El propio proyecto lo marca con advertencia: puede generar un loop largo entre Claude y Codex y drenar tu cuota de uso rápido. Conviene dejarlo apagado y correr las revisiones a mano, salvo que vayas a quedarte viendo esa sesión de cerca.

SECCIÓN 08
BUENAS PRÁCTICAS

No revises cada commit. Revisá antes de entregar.

- Corré `/codex:review` en segundo plano (`--background`) en cambios de varios archivos: puede tardar, y bloquear la sesión esperando no suma nada.
- Guardá `/codex:adversarial-review` para lo que de verdad importa: autenticación, dinero, borrado de datos, migraciones. Ahí sí vale la pena que cuestione el diseño, no solo el código.
- Para `/codex:rescue` , usá un modelo liviano (Luna o spark) en tareas chicas, y guardá el modelo grande para lo que de verdad lo necesita: el costo escala rápido.
- No reemplaza la revisión humana. La complementa: es una segunda mirada con otro entrenamiento, no un reemplazo del criterio de alguien que conoce el proyecto.

SECCIÓN 09
EL ERROR MÁS COMÚN

Activar el gate automático y dejar la sesión sola.

Es la forma más rápida de quedarte sin cuota en ChatGPT o en tu API key de OpenAI en una sola tarde, porque cada respuesta de Claude dispara una revisión de Codex, y si algo no cierra, la sesión queda atascada corrigiendo en loop hasta que alguien la mira. Empezá siempre con revisiones manuales, y activá el gate solo en una sesión que vayas a estar vigilando de verdad.

CÓMO LO USO YO

En Neiro Agency lo tengo instalado hace poco y lo uso puntual, no en cada commit. Antes de entregarle algo a un cliente (un workflow de n8n con lógica compleja, o un cambio en la integración de WhatsApp Business) corro un `/codex:review --background` mientras sigo con otra cosa, y reviso el resultado cuando termina. Es una segunda mirada barata antes de mandar algo a producción con dinero o datos de un cliente de por medio.

El `/codex:adversarial-review` lo reservo para lo que puede salir caro si falla: credenciales, permisos, cualquier flujo que toque plata o borre algo. El gate automático lo tengo desactivado: con la cuota que gasto en Claude Code y en Codex por separado, no me interesa un loop corriendo solo mientras hago otra cosa.

SIGUIENTE PASO

Instalá el plugin hoy con los cuatro comandos de esta guía, corré `/codex:setup`, y probalo en tu próximo cambio antes de entregarlo: `/codex:review --background`, y revisá el resultado con `/codex:result`. Dejá el gate automático apagado hasta que le agarrés el pulso al consumo de cuota.